

# 1. 📁 Core Process Concepts & Structure

---

Question 1:

Describe the various states in a **Process Control Block (PCB)**.

Solution:

The **Process Control Block (PCB)**, also known as a task control block, is a data structure maintained by the Operating System for **every process**. It acts as a repository for all the information required by the OS to manage and control a process.

The essential categories of information stored in a PCB include:

1. **Process State**: The current state of the process (e.g., New, Ready, Running, Waiting/Blocked, Terminated).
2. **Program Counter (PC)**: Indicates the address of the next instruction to be executed for this process.
3. **CPU Registers**: Contents of all CPU registers (e.g., data registers, index registers, stack pointers) used by the process when it was last running. These are crucial for context switching.
4. **CPU Scheduling Information**: Process priority, pointers to scheduling queues, and other scheduling parameters.
5. **Memory Management Information**: Base and limit registers, page tables, or segment tables, depending on the memory system used.
6. **Accounting Information**: CPU time used, time limits, job or process numbers, and overall elapsed time.
7. **I/O Status Information**: List of I/O devices allocated to the process, a list of open files, and any pending I/O requests.



Question 2:

Explain the structure of a **Linux process management system**.

Solution:

In Linux, process management centers around the concept of a **Process** and the use of the

`fork()` and `exec()` system calls. The entire process structure is maintained using a data structure called `task_struct`.

- **`task_struct`**: This is the equivalent of the PCB in Linux. It contains all the process-specific information, including state, scheduling details, PID, memory pointers, open files, and parent/child relationships.
- **Process Creation ( `fork()` )**: The `fork()` system call creates a new process (the **child**) that is an almost exact copy of the calling process (the **parent**). The child process gets a copy of the parent's address space, but they have separate PID's.
- **Program Execution ( `exec()` )**: The `exec()` family of calls is used to load a new program into the existing process's memory space, replacing the old program's code, data, and stack. This is typically called immediately after `fork()` to execute a new application.
- **Threads (Lightweight Processes - LWP)**: Linux implements threads as **Lightweight Processes (LWP)**. Threads share the same memory space but have their own `task_struct` and PID (or more accurately, a shared *Thread Group ID* but a unique *Process ID* in modern kernels). This allows them to be managed efficiently by the scheduler.
- **The Process Tree**: All processes in Linux form a hierarchy, or process tree. The initial process, `init` (PID 1), is the root of this tree. Every process has a parent (except `init`), and the `task_struct` maintains pointers for parent, children, and siblings.

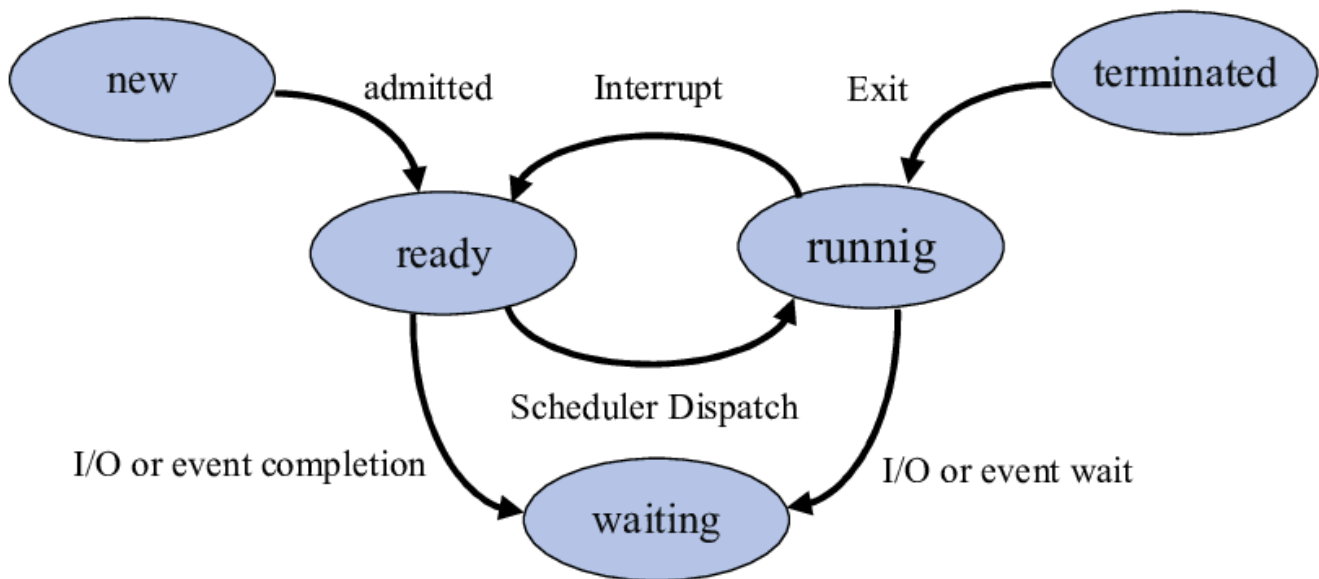


Question 3:

Describe the **Process Transition Diagram** with a labeled diagram.

Solution:

The **Process Transition Diagram** illustrates the various states a process can transition through during its lifetime, managed by the Operating System.



The key states and transitions are:

1. **New**: The process is being created. It is not yet loaded into main memory.
  - Transition: New → Ready (When the OS admits the process and allocates initial resources).
2. **Ready**: The process is loaded into main memory and is waiting for the CPU. It is eligible to run.
  - Transition: Ready → Running (The CPU scheduler selects the process; **Dispatch**).
3. **Running**: The process is currently executing instructions on the CPU.
  - Transition 1: Running → Ready (When the time slice expires or a higher-priority process arrives; **Interrupt/Time-out**).
  - Transition 2: Running → Waiting (When the process requests an I/O operation or waits for an event; **I/O or Event Wait**).
  - Transition 3: Running → Terminated (When the process finishes execution or is explicitly killed).
4. **Waiting (Blocked)**: The process is waiting for some event to occur, such as the completion of an I/O operation or the availability of a resource. It cannot run even if the CPU is free.
  - Transition: Waiting → Ready (When the I/O operation completes or the requested event occurs).
5. **Terminated (Halt)**: The process has finished execution. The OS typically holds its state temporarily for accounting purposes before deallocating resources.

## 2. 🕒 CPU Scheduling: Theory & Comparison

---

Question 1:

Discuss the different types of **process scheduling queues**.

Solution:

Process scheduling in an OS involves moving processes through several queues representing their current state. The three main types of process scheduling queues are:

### 1. **Job Queue (or Batch Queue):**

- **Function:** This queue holds all processes that are currently in the system, waiting to be allocated memory and moved into the ready state.
- **Scheduler:** Managed by the **Long-Term Scheduler** (or Job Scheduler), which controls the degree of multiprogramming.
- **State:** Processes here are typically in the **New** or **Waiting** (for memory) state.

### 2. **Ready Queue:**

- **Function:** This queue holds all processes that reside in **main memory**, are **ready** and waiting to execute, and are simply waiting for the CPU to become available.
- **Scheduler:** Managed by the **Short-Term Scheduler** (or CPU Scheduler).
- **Structure:** Often implemented as a linked list of Process Control Blocks (PCBs). [Image of the process state diagram showing Ready, Running, Waiting, New, and Terminated states with queues and transitions]

### 3. **Device (or I/O) Queue:**

- **Function:** This queue holds processes that are **waiting** for a particular I/O device to complete its request (e.g., reading from a disk, waiting for network data).
- **Scheduler:** Managed by the **Medium-Term Scheduler** (when swapping is involved) or the **I/O Subsystem**.
- **State:** Processes here are in the **Waiting** (or Blocked) state, specifically blocked on an I/O event. Each device generally has its own dedicated queue.



Question 2:

Difference between **Multilevel Queue** and **Multilevel Feedback Queue Scheduling**.

Solution:

Both are advanced scheduling algorithms that partition the ready queue into multiple separate queues, but they differ fundamentally in how processes move between these queues.

| Feature          | Multilevel Queue Scheduling (MLQ)                                                                                                                      | Multilevel Feedback Queue Scheduling (MLFQ)                                                                                                                                                      |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Process Movement | <b>No movement</b> between queues. Processes are permanently assigned to a queue based on a characteristic (e.g., system vs. interactive).             | <b>Allows movement</b> between queues. Processes can migrate to lower-priority queues if they use too much CPU time (I/O bound) or move to higher-priority queues if they wait too long (Aging). |
| Flexibility      | <b>Less flexible</b> . Permanent assignment can lead to poor performance if a process changes behavior (e.g., an I/O-bound process becomes CPU-bound). | <b>Highly flexible</b> . Dynamically adjusts priority based on the observed behavior of the process, improving responsiveness.                                                                   |
| Starvation       | Can be a <b>major issue</b> . If a high-priority queue is never empty, low-priority processes may never run.                                           | Designed to <b>prevent starvation</b> using a mechanism called <b>Aging</b> (moving long-waiting processes to higher-priority queues).                                                           |
| Complexity       | <b>Simpler</b> to implement, as the assignment is static.                                                                                              | <b>More complex</b> to implement due to the need to define parameters for queue switching (e.g., time quantum for each queue, rules for demotion/promotion).                                     |
| Common Use       | Often used for system processes (high priority) vs. user processes (low priority).                                                                     | Used in many modern interactive operating systems (e.g., UNIX, Linux kernels) to achieve a good balance of responsiveness and throughput.                                                        |



Question 3:

Explain **Round Robin Scheduling** with an example.

Solution:

**Round Robin (RR) Scheduling** is a preemptive scheduling algorithm designed primarily for time-sharing and interactive systems to ensure **fairness** and **responsiveness**.

## Explanation

1. **Time Quantum ( $q$ ):** RR assigns a small unit of time, called a **time quantum** ( $q$ ) or time slice, to each process. Typical quantum values range from 10 to 100 milliseconds.
2. **Cyclic Execution:** The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a duration of at most one time quantum.
3. **Preemption:**
  - If a process finishes its burst within the quantum, it releases the CPU voluntarily.
  - If the process is still running when the quantum expires, the CPU is **preempted** (taken away), and the process is moved to the **tail** of the ready queue. The CPU is then given to the next process at the head of the queue.

## Example

---

Consider three processes (P1, P2, P3) arriving at time 0 with the following CPU burst times and a time quantum ( $q$ ) of **4 milliseconds (ms)**:

| Process | Burst Time (ms) |
|---------|-----------------|
| P1      | 10              |
| P2      | 5               |
| P3      | 3               |

### Gantt Chart:

| P1 (4) | P2 (4) | P3 (3) | P1 (4) | P2 (1) | P1 (2) |
|--------|--------|--------|--------|--------|--------|
| 0      | 4      | 8      | 11     | 15     | 16     |

- **0-4 (P1):** P1 runs for 4ms. Remaining burst is 6ms. P1 goes to the end of the queue.
- **4-8 (P2):** P2 runs for 4ms. Remaining burst is 1ms. P2 goes to the end of the queue.
- **8-11 (P3):** P3 runs for 3ms. P3 finishes and exits.
- **11-15 (P1):** P1 runs for 4ms. Remaining burst is 2ms. P1 goes to the end of the queue.
- **15-16 (P2):** P2 runs for its remaining 1ms. P2 finishes and exits.
- **16-18 (P1):** P1 runs for its remaining 2ms. P1 finishes and exits.

**Total Completion Time (Throughput):** 18 ms. **Average Waiting Time:**

$$(8 - 4) + (11 - 4) + (4 + 15 - 8)/3 = 4 + 7 + 11/3 \approx 7.33 \text{ ms.}$$

Question 4:

Explain the concept of **starvation** in scheduling and methods to handle it.

Solution:

## Concept of Starvation

---

**Starvation** is a critical problem in concurrent systems where a process (or thread) is perpetually denied access to a resource it needs (like the CPU, a lock, or an I/O device), even though the resource is available.

- **Cause:** Starvation typically occurs in scheduling algorithms that heavily favor higher-priority or shorter jobs, such as **Priority Scheduling** or strict **Shortest Job First (SJF)**. If a stream of high-priority jobs continuously arrives, a low-priority job may never get a chance to run.
- **Effect:** The starving process appears to be in an infinite loop of waiting, leading to effective non-completion, which severely degrades system performance and fairness for that specific task.

## Methods to Handle Starvation

---

The primary method used to prevent or solve starvation is **Aging**:

- **Aging:** This technique involves **gradually increasing the priority** of processes that have been waiting in the ready queue for a long time.
  - **Mechanism:** For every time unit a process waits, its priority is increased by a small, predetermined amount.
  - **Effect:** Eventually, the starving process's priority will rise high enough to surpass the priority of all newly arriving processes, guaranteeing that it will eventually be scheduled and executed.

Another less common or specialized method is:

- **Guarantee Scheduling:** The OS makes guarantees about the percentage of CPU time a process will receive. If a process falls below its guaranteed allocation, its priority is boosted.

Question 5:  
Compare and contrast SJF and Round Robin scheduling with examples.

Solution:  
Shortest Job First (SJF) and Round Robin (RR) are two fundamental scheduling algorithms that prioritize different metrics: efficiency (SJF) versus fairness/responsiveness (RR).

## Comparison and Contrast

| Feature        | Shortest Job First (SJF)                                                                                | Round Robin (RR)                                                         |
|----------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Goal           | Optimal throughput and minimum average waiting time.                                                    | Fair allocation of the CPU and good response time for interactive users. |
| Preemption     | Can be non-preemptive (once started, runs to completion) or preemptive (Shortest Remaining Time First). | Always preemptive based on the time quantum ( $q$ ).                     |
| Fairness       | Unfair. Heavily favors short jobs; long jobs can starve.                                                | Fair. Each process gets an equal share of CPU time in a cycle.           |
| Overhead       | Low. Context switching only occurs when a process terminates or blocks (non-preemptive).                | High. Requires frequent context switching after every quantum $q$ .      |
| Predictability | Difficult to implement in real systems, as predicting the exact next CPU burst time is impossible.      | Predictable once the quantum $q$ is set.                                 |

## Illustrative Example (Same processes as Q3)

Processes (P1: 10ms, P2: 5ms, P3: 3ms). Arrival Time (AT) = 0.

| Algorithm            | Gantt Chart               | Average Waiting Time                                                        | Key Observation                                                                                |
|----------------------|---------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| SJF (Non-Preemptive) | P3 (3)   P2 (5)   P1 (10) | Waiting Times: P3 (0), P2 (3), P1 (8). AWT: $(0 + 3 + 8)/3 = 3.67\text{ms}$ | SJF achieves the absolute minimum AWT by prioritizing the shortest job (P3, then P2, then P1). |



| Algorithm               | Gantt Chart                                         | Average Waiting Time                                                        | Key Observation                                                                                     |
|-------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Round Robin ( $q = 4$ ) | P1 (4)   P2 (4)   P3 (3)   P1 (4)   P2 (1)   P1 (2) | Waiting Times: P1 (8), P2 (7), P3 (0). AWT: $(8 + 7 + 0)/3 = 5.00\text{ms}$ | RR ensures P1 gets a turn early, improving fairness but increasing the overall AWT compared to SJF. |



## CPU Scheduling Calculations Problems

### Fresh Dataset for Calculations (FCFS & SJF)

To cover various scenarios and make the problems challenging, the dataset will feature different Arrival Times (ATs) and a mix of Burst Times (BTs).

| Process | Arrival Time (AT) | Burst Time (BT) |
|---------|-------------------|-----------------|
| P1      | 0                 | 10              |
| P2      | 2                 | 5               |
| P3      | 3                 | 2               |
| P4      | 6                 | 4               |



Question 9:  
Consider the following set of processes with their respective arrival times (AT) and burst times (BT), scheduled using the **First-Come, First-Served (FCFS) algorithm**... Calculate: The completion time (CT) for each process, the average waiting time (AWT), and the average turnaround time (TAT). Show your calculations.

**Dataset Used:** P1(0, 10), P2(2, 5), P3(3, 2), P4(6, 4).

Solution:

## FCFS Calculation

---

**Execution Order:** P1, P2, P3, P4 (Strictly by Arrival Time)

| Process | AT | BT | Completion Time (CT) | Turnaround Time (TAT) | Waiting Time (WT) |
|---------|----|----|----------------------|-----------------------|-------------------|
| P1      | 0  | 10 | $0 + 10 = 10$        | $10 - 0 = 10$         | $10 - 10 = 0$     |
| P2      | 2  | 5  | $10 + 5 = 15$        | $15 - 2 = 13$         | $13 - 5 = 8$      |
| P3      | 3  | 2  | $15 + 2 = 17$        | $17 - 3 = 14$         | $14 - 2 = 12$     |
| P4      | 6  | 4  | $17 + 4 = 21$        | $21 - 6 = 15$         | $15 - 4 = 11$     |
| Sum     |    |    |                      | 52                    | 31                |

### Calculations:

- **Total TAT** =  $10 + 13 + 14 + 15 = 52$
- **Total WT** =  $0 + 8 + 12 + 11 = 31$
- **Average Turnaround Time (ATAT)** =  $\frac{52}{4} = 13.0$  units.
- **Average Waiting Time (AWT)** =  $\frac{31}{4} = 7.75$  units.

---

∞

---

Question 10:

Consider the following set of processes... The scheduling follows the **First-Come, First-Served (FCFS) algorithm**. Tasks: Construct a **Gantt Chart**, Calculate the Completion Time (CT), Determine the Turnaround Time (TAT) and Waiting Time (WT), Compute the Average Waiting Time (AWT) and Average Turnaround Time (ATAT), and Interpret the results.

Solution:

*The calculations are identical to Q9.*

## Gantt Chart (FCFS)

---

| P1 | P2 | P3 | P4 |    |
|----|----|----|----|----|
| 0  | 10 | 15 | 17 | 21 |

## Calculation Summary

---

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| P1      | 0  | 10 | 10 | 10  | 0  |
| P2      | 2  | 5  | 15 | 13  | 8  |
| P3      | 3  | 2  | 17 | 14  | 12 |
| P4      | 6  | 4  | 21 | 15  | 11 |

### Averages:

- **Average Turnaround Time (ATAT): 13.0** units.
- **Average Waiting Time (AWT): 7.75** units.

## Interpretation of Results

---

FCFS is simple but suffers from the **Convoy Effect**. Here, the small jobs P3 (BT=2) and P4 (BT=4) must wait until  $t = 15$  and  $t = 17$ , respectively, because they are stuck behind the long initial job P1 (BT=10). This results in high waiting times (12 and 11 units) for the quickly executable processes, increasing the overall average waiting time.



Question 11:

Using **FCFS**, calculate for each process: Completion Time (CT), Turnaround Time ( $TAT = CT - AT$ ), and Waiting Time ( $WT = TAT - BT$ ). Also calculate: ATAT, AWT, ACT.

Solution:

*The calculations are identical to Q9 and Q10.*

## FCFS Calculation Summary

---

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| P1      | 0  | 10 | 10 | 10  | 0  |
| P2      | 2  | 5  | 15 | 13  | 8  |
| P3      | 3  | 2  | 17 | 14  | 12 |
| P4      | 6  | 4  | 21 | 15  | 11 |
| Sum     |    |    | 63 | 52  | 31 |

### Calculations:

- **Average Turnaround Time (ATAT):**  $\frac{52}{4} = 13.0$  units.
- **Average Waiting Time (AWT):**  $\frac{31}{4} = 7.75$  units.
- **Average Completion Time (ACT):**  $\frac{\text{Total CT}}{\text{No. of Processes}} = \frac{63}{4} = 15.75$  units.

---

»

---

Question 18:

Solve for the average waiting time and turnaround time using the **First Come First Serve (FCFS) algorithm** for the following processes:

Solution:

*The calculations are identical to Q9, Q10, and Q11.*

## FCFS Calculation Summary

---

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| P1      | 0  | 10 | 10 | 10  | 0  |
| P2      | 2  | 5  | 15 | 13  | 8  |

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| P3      | 3  | 2  | 17 | 14  | 12 |
| P4      | 6  | 4  | 21 | 15  | 11 |

### Results:

- **Average Turnaround Time (ATAT): 13.0** units.
- **Average Waiting Time (AWT): 7.75** units.

»

Question 12:

Use **SJF (non-preemptive)** to calculate for each process: CT, TAT, WT. Also calculate: ATAT, AWT, ACT.

Solution:

## SJF (Non-Preemptive) Calculation

### Execution Logic:

1. **t=0**: Only **P1** is available (BT=10). P1 starts.
2. **P1 runs** until it completes (Non-preemptive).
3. **t=10**: P1 finishes. All other processes (P2, P3, P4) have arrived. Available BTs: P2(5), P3(2), P4(4).
4. **Order (SJF at t=10)**: P3 (shortest), P4, P2 (longest).

### Gantt Chart (SJF Non-Preemptive):

| P1 | P3 | P4 | P2 |    |
|----|----|----|----|----|
| 0  | 10 | 12 | 16 | 21 |

| Process | AT | BT | CT | TAT (CT - AT) | WT (TAT - BT) |
|---------|----|----|----|---------------|---------------|
| P1      | 0  | 10 | 10 | $10 - 0 = 10$ | $10 - 10 = 0$ |
| P2      | 2  | 5  | 21 | $21 - 2 = 19$ | $19 - 5 = 14$ |

| Process | AT | BT | CT | TAT (CT - AT) | WT (TAT - BT) |
|---------|----|----|----|---------------|---------------|
| P3      | 3  | 2  | 12 | $12 - 3 = 9$  | $9 - 2 = 7$   |
| P4      | 6  | 4  | 16 | $16 - 6 = 10$ | $10 - 4 = 6$  |
| Sum     |    |    | 59 | 48            | 27            |

### Calculations:

- **Total TAT** =  $10 + 19 + 9 + 10 = 48$
- **Total WT** =  $0 + 14 + 7 + 6 = 27$
- **Average Turnaround Time (ATAT)**:  $\frac{48}{4} = 12.0$  units.
- **Average Waiting Time (AWT)**:  $\frac{27}{4} = 6.75$  units.
- **Average Completion Time (ACT)**:  $\frac{59}{4} = 14.75$  units.



To cover the logic for **SJF Non-Preemptive, SRTF, and Round Robin (RR)**, I will use a **new, single, challenging dataset** for these questions.

## New Dataset for Calculations (SJF, SRTF, RR)

This dataset is designed to challenge the preemption logic of SRTF and the scheduling decisions of SJF.

| Process | Arrival Time (AT) | Burst Time (BT) |
|---------|-------------------|-----------------|
| P1      | 0                 | 8               |
| P2      | 1                 | 4               |
| P3      | 2                 | 1               |
| P4      | 3                 | 4               |
| P5      | 5                 | 5               |

Question 19:

Compute the average turnaround time using the **Shortest Job First (SJF) non-preemptive algorithm** for these processes:

Solution:

## SJF (Non-Preemptive) Calculation

---

### Execution Logic:

1. **t=0**: Only **P1** (BT=8) is available. P1 starts.
2. **P1 runs** until  $t = 8$  (Non-preemptive).
3. **t=8**: P1 finishes. All other processes (P2, P3, P4, P5) have arrived. Available BTs: P2(4), P3(1), P4(4), P5(5).
4. **Order (SJF at t=8)**: P3 (shortest), P2/P4 (tie, choose P2 first), P4, P5 (longest).

### Gantt Chart (SJF Non-Preemptive):

| P1 | P3 | P2 | P4 | P5 |
|----|----|----|----|----|
| 0  | 8  | 9  | 13 | 17 |

| Process | AT | BT | CT | Turnaround Time (TAT = CT - AT) | Waiting Time (WT = TAT - BT) |
|---------|----|----|----|---------------------------------|------------------------------|
| P1      | 0  | 8  | 8  | $8 - 0 = 8$                     | $8 - 8 = 0$                  |
| P2      | 1  | 4  | 13 | $13 - 1 = 12$                   | $12 - 4 = 8$                 |
| P3      | 2  | 1  | 9  | $9 - 2 = 7$                     | $7 - 1 = 6$                  |
| P4      | 3  | 4  | 17 | $17 - 3 = 14$                   | $14 - 4 = 10$                |
| P5      | 5  | 5  | 22 | $22 - 5 = 17$                   | $17 - 5 = 12$                |
| Sum     |    |    |    | 58                              | 36                           |

### Result:

- **Average Turnaround Time (ATAT)**:  $\frac{58}{5} = 11.6$  units.

---

∞

---

Question 14:

Use **Preemptive SJF (SRTF)** to calculate CT, TAT, WT, ATAT, AWT, ACT.

Solution:

## SRTF (Preemptive SJF) Calculation

### Execution Logic:

| Time | Event      | Ready Queue (Remaining BT)   | Execution                              |
|------|------------|------------------------------|----------------------------------------|
| 0    | P1 arrives | (P1: 8)                      | P1 starts                              |
| 1    | P2 arrives | (P1: 7), P2: 4               | P2 preempts P1 (4 < 7)                 |
| 2    | P3 arrives | (P2: 3), P3: 1, P1: 7        | P3 preempts P2 (1 < 3)                 |
| 3    | P4 arrives | (P3: 0), P4: 4, P2: 3, P1: 7 | P3 finishes. P2 starts (3 is smallest) |
| 5    | P5 arrives | (P2: 1), P5: 5, P4: 4, P1: 7 | P2 continues (1 is smallest)           |
| 6    |            | (P2: 0), P5: 5, P4: 4, P1: 7 | P2 finishes. P4 starts (4 is smallest) |
| 10   |            | P5: 5, P1: 7, (P4: 0)        | P4 finishes. P5 starts (5 < 7)         |
| 15   |            | P1: 7, (P5: 0)               | P5 finishes. P1 starts                 |
| 22   |            | (P1: 0)                      | P1 finishes                            |

### Gantt Chart (SRTF):

| P1 | P2 | P3 | P2 | P4 | P5 | P1 |
|----|----|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 6  | 10 | 15 |

| Process | AT | BT | CT | TAT (CT - AT) | WT (TAT - BT) |
|---------|----|----|----|---------------|---------------|
| P1      | 0  | 8  | 22 | $22 - 0 = 22$ | $22 - 8 = 14$ |
| P2      | 1  | 4  | 6  | $6 - 1 = 5$   | $5 - 4 = 1$   |
| P3      | 2  | 1  | 3  | $3 - 2 = 1$   | $1 - 1 = 0$   |
| P4      | 3  | 4  | 10 | $10 - 3 = 7$  | $7 - 4 = 3$   |
| P5      | 5  | 5  | 15 | $15 - 5 = 10$ | $10 - 5 = 5$  |
| Sum     |    |    | 56 | 45            | 23            |

### Averages:

- Average Turnaround Time (ATAT):  $\frac{45}{5} = 9.0$  units.



- **Average Waiting Time (AWT):**  $\frac{23}{5} = 4.6$  units.
- **Average Completion Time (ACT):**  $\frac{56}{5} = 11.2$  units.



Question 20:

Analyze the response time and waiting time using **Shortest Remaining Time First (SRTF)** for the given dataset:

Solution:

*The calculations are based on the SRTF table from Q14.*

| Process | AT | BT | CT | WT (from Q14) | Response Time (RT = First Start Time - AT) |
|---------|----|----|----|---------------|--------------------------------------------|
| P1      | 0  | 8  | 22 | 14            | $0 - 0 = 0$                                |
| P2      | 1  | 4  | 6  | 1             | $1 - 1 = 0$                                |
| P3      | 2  | 1  | 3  | 0             | $2 - 2 = 0$                                |
| P4      | 3  | 4  | 10 | 3             | $6 - 3 = 3$                                |
| P5      | 5  | 5  | 15 | 5             | $10 - 5 = 5$                               |
| Sum     |    |    |    | 23            | 8                                          |

### Analysis of Waiting Time and Response Time:

- **Waiting Time (WT):** Calculated as (Time spent waiting in the ready queue). It is the total time a process spends waiting for the CPU.
- **Response Time (RT):** Calculated as (Time of first dispatch) – (Arrival Time). It is the initial delay a user experiences before their job begins processing.

### Results:

- **Average Waiting Time (AWT):**  $\frac{23}{5} = 4.6$  units.
- **Average Response Time (ART):**  $\frac{8}{5} = 1.6$  units.



Question 1:

Use the **Round Robin scheduling algorithm** to schedule five processes and compute the turnaround time.

Solution:

I will use the same dataset and assume a reasonable **Time Quantum (Q=3)** for this general RR question.

## Round Robin (RR) Calculation (Quantum Q=3)

---

**Execution Sequence:**

| Time      | Ready Queue        | Process Run                       | Remaining BT |
|-----------|--------------------|-----------------------------------|--------------|
| 0         | (P1)               | <b>P1</b> (3)                     | P1: 5        |
| 3         | P2, P3, P4, P1     | <b>P2</b> (3)                     | P2: 1        |
| 6         | P3, P4, P1, P5, P2 | <b>P3</b> (1 - <b>Finishes!</b> ) | P3: 0        |
| 7         | P4, P1, P5, P2     | <b>P4</b> (3)                     | P4: 1        |
| 10        | P1, P5, P2, P4     | <b>P1</b> (3)                     | P1: 2        |
| 13        | P5, P2, P4, P1     | <b>P5</b> (3)                     | P5: 2        |
| 16        | P2, P4, P1, P5     | <b>P2</b> (1 - <b>Finishes!</b> ) | P2: 0        |
| 17        | P4, P1, P5         | <b>P4</b> (1 - <b>Finishes!</b> ) | P4: 0        |
| 18        | P1, P5             | <b>P1</b> (2 - <b>Finishes!</b> ) | P1: 0        |
| 20        | P5                 | <b>P5</b> (2 - <b>Finishes!</b> ) | P5: 0        |
| <b>22</b> |                    |                                   |              |

**Gantt Chart (RR, Q=3):**

| <b>P1</b> | <b>P2</b> | <b>P3</b> | <b>P4</b> | <b>P1</b> | <b>P5</b> | <b>P2</b> | <b>P4</b> | <b>P1</b> | <b>P5</b> |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0         | 3         | 6         | 7         | 10        | 13        | 16        | 17        | 18        | 20        |

| Process | AT | BT | CT        | Turnaround Time (TAT) |
|---------|----|----|-----------|-----------------------|
| P1      | 0  | 8  | <b>20</b> | $20 - 0 = 20$         |
| P2      | 1  | 4  | <b>17</b> | $17 - 1 = 16$         |
| P3      | 2  | 1  | <b>7</b>  | $7 - 2 = 5$           |

| Process | AT | BT | CT | Turnaround Time (TAT) |
|---------|----|----|----|-----------------------|
| P4      | 3  | 4  | 18 | $18 - 3 = 15$         |
| P5      | 5  | 5  | 22 | $22 - 5 = 17$         |
| Sum     |    |    |    | 73                    |

Result:

- **Total Turnaround Time (TAT): 73 units.**
- **Average Turnaround Time (ATAT):**  $\frac{73}{5} = 14.6$  units.

»

Question 13:

Time Quantum: 3 milliseconds. Draw the **Gantt Chart**, then compute: CT, TAT, WT, Averages (ATAT, AWT, ACT).

Solution:

*The calculations are identical to Q1.*

## Round Robin (RR) Calculation (Quantum Q=3)

**Gantt Chart (RR, Q=3):**

| P1 | P2 | P3 | P4 | P1 | P5 | P2 | P4 | P1 | P5 |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 0  | 3  | 6  | 7  | 10 | 13 | 16 | 17 | 18 | 20 | 22 |

| Process | AT | BT | CT | TAT (CT - AT) | WT (TAT - BT) |
|---------|----|----|----|---------------|---------------|
| P1      | 0  | 8  | 20 | 20            | $20 - 8 = 12$ |
| P2      | 1  | 4  | 17 | 16            | $16 - 4 = 12$ |
| P3      | 2  | 1  | 7  | 5             | $5 - 1 = 4$   |
| P4      | 3  | 4  | 18 | 15            | $15 - 4 = 11$ |
| P5      | 5  | 5  | 22 | 17            | $17 - 5 = 12$ |
| Sum     |    |    | 74 | 73            | 51            |

### Averages:

- **Average Turnaround Time (ATAT):**  $\frac{73}{5} = 14.6$  units.
- **Average Waiting Time (AWT):**  $\frac{51}{5} = 10.2$  units.
- **Average Completion Time (ACT):**  $\frac{74}{5} = 14.8$  units.

For the **Priority Scheduling** questions (Q15, Q17, Q21) and the final **RR** question (Q16), I will use the following comprehensive dataset, adding Priority where required.

### Dataset for Calculations (RR & Priority)

---

This dataset is designed to test priority and time quantum decisions effectively.

| Process | Arrival Time (AT) | Burst Time (BT) | Priority (P) (Lower = Higher) |
|---------|-------------------|-----------------|-------------------------------|
| P1      | 0                 | 7               | 3 (Medium)                    |
| P2      | 1                 | 4               | 1 (Highest)                   |
| P3      | 3                 | 2               | 4 (Low)                       |
| P4      | 4                 | 5               | 2 (High-Med)                  |

Question 16:

Time Quantum: 2 milliseconds. Task: Draw the **Gantt Chart** and compute CT, TAT, WT, ATAT, AWT, ACT.

Solution:

### Round Robin (RR) Calculation (Quantum Q=2)

---

#### Execution Sequence:

| Time | Event          | Ready Queue (Remaining BT)              | Execution |
|------|----------------|-----------------------------------------|-----------|
| 0    | (P1)           | <b>P1</b> (2 units)                     | P1: 5     |
| 2    | P2, P3, P1     | <b>P2</b> (2 units)                     | P2: 2     |
| 4    | P3, P1, P4, P2 | <b>P3</b> (2 units - <b>Finishes!</b> ) | P3: 0     |
| 6    | P1, P4, P2     | <b>P1</b> (2 units)                     | P1: 3     |
| 8    | P4, P2, P1     | <b>P4</b> (2 units)                     | P4: 3     |

| Time      | Event      | Ready Queue (Remaining BT)              | Execution |
|-----------|------------|-----------------------------------------|-----------|
| 10        | P2, P1, P4 | <b>P2</b> (2 units - <b>Finishes!</b> ) | P2: 0     |
| 12        | P1, P4     | <b>P1</b> (2 units)                     | P1: 1     |
| 14        | P4, P1     | <b>P4</b> (2 units)                     | P4: 1     |
| 16        | P1, P4     | <b>P1</b> (1 unit - <b>Finishes!</b> )  | P1: 0     |
| 17        | P4         | <b>P4</b> (1 unit - <b>Finishes!</b> )  | P4: 0     |
| <b>18</b> |            |                                         |           |

### Gantt Chart (RR, Q=2):

| P1 | P2 | P3 | P1 | P4 | P2 | P1 | P4 | P1 | P4 |
|----|----|----|----|----|----|----|----|----|----|
| 0  | 2  | 4  | 6  | 8  | 10 | 12 | 14 | 16 | 17 |

| Process    | AT | BT | CT        | TAT (CT - AT) | WT (TAT - BT) |
|------------|----|----|-----------|---------------|---------------|
| P1         | 0  | 7  | <b>17</b> | $17 - 0 = 17$ | $17 - 7 = 10$ |
| P2         | 1  | 4  | <b>12</b> | $12 - 1 = 11$ | $11 - 4 = 7$  |
| P3         | 3  | 2  | <b>6</b>  | $6 - 3 = 3$   | $3 - 2 = 1$   |
| P4         | 4  | 5  | <b>18</b> | $18 - 4 = 14$ | $14 - 5 = 9$  |
| <b>Sum</b> |    |    | <b>53</b> | <b>45</b>     | <b>27</b>     |

### Averages:

- **Average Turnaround Time (ATAT):**  $\frac{45}{4} = 11.25$  units.
- **Average Waiting Time (AWT):**  $\frac{27}{4} = 6.75$  units.
- **Average Completion Time (ACT):**  $\frac{53}{4} = 13.25$  units.

»

Question 15:

**Note:** Lower number = higher priority. Task: Schedule the processes using **non-preemptive priority scheduling**. Calculate CT, TAT, WT, ATAT, AWT, ACT.

Solution:

## Priority (Non-Preemptive) Calculation (Lower number = Higher Priority)

---

### Execution Logic:

1. **t=0**: Only **P1** (P=3) is available. P1 starts.
2. **P1 runs** until  $t = 7$  (Non-preemptive).
3. **t=7**: P1 finishes. All other processes (P2, P3, P4) are available. Priorities: P2(1-Highest), P4(2), P1(3), P3(4-Lowest).
4. **Order (at t=7)**: P2, P4, P3.

### Gantt Chart (Priority Non-Preemptive):

| P1 | P2 | P4 | P3 |
|----|----|----|----|
| 0  | 7  | 11 | 16 |

| Process | AT | BT | Priority | CT | TAT (CT - AT) | WT (TAT - BT) |
|---------|----|----|----------|----|---------------|---------------|
| P1      | 0  | 7  | 3        | 7  | $7 - 0 = 7$   | $7 - 7 = 0$   |
| P2      | 1  | 4  | 1        | 11 | $11 - 1 = 10$ | $10 - 4 = 6$  |
| P3      | 3  | 2  | 4        | 18 | $18 - 3 = 15$ | $15 - 2 = 13$ |
| P4      | 4  | 5  | 2        | 16 | $16 - 4 = 12$ | $12 - 5 = 7$  |
| Sum     |    |    |          | 52 | 44            | 26            |

### Averages:

- **Average Turnaround Time (ATAT)**:  $\frac{44}{4} = 11.0$  units.
- **Average Waiting Time (AWT)**:  $\frac{26}{4} = 6.5$  units.
- **Average Completion Time (ACT)**:  $\frac{52}{4} = 13.0$  units.



Question 17:

Consider the set of 5 processes... If the CPU scheduling policy is **priority non-preemptive** (Higher number represents higher priority), calculate the average waiting time and average turnaround time.

Solution:

## Priority (Non-Preemptive) Calculation (Higher number = Higher Priority)

---

**Priority Mapping (Reversed):** P3(4-Highest), P1(3), P4(2), P2(1-Lowest).

### Execution Logic:

1. **t=0:** Only **P1** (P=3) is available. P1 starts.
2. **P1 runs** until  $t = 7$  (Non-preemptive).
3. **t=7:** P1 finishes. All other processes (P2, P3, P4) are available. Priorities (High to Low): P3(4), P1(3), P4(2), P2(1).
4. **Order (at t=7):** P3, P4, P2.

### Gantt Chart (Priority Non-Preemptive - Higher=Higher):

| P1 | P3 | P4 | P2 |
|----|----|----|----|
| 0  | 7  | 9  | 14 |

| Process | AT | BT | Priority | CT | TAT (CT - AT) | WT (TAT - BT) |
|---------|----|----|----------|----|---------------|---------------|
| P1      | 0  | 7  | 3        | 7  | $7 - 0 = 7$   | $7 - 7 = 0$   |
| P2      | 1  | 4  | 1        | 18 | $18 - 1 = 17$ | $17 - 4 = 13$ |
| P3      | 3  | 2  | 4        | 9  | $9 - 3 = 6$   | $6 - 2 = 4$   |
| P4      | 4  | 5  | 2        | 14 | $14 - 4 = 10$ | $10 - 5 = 5$  |
| Sum     |    |    |          | 48 | 40            | 22            |

### Averages:

- **Average Turnaround Time (ATAT):**  $\frac{40}{4} = 10.0$  units.
- **Average Waiting Time (AWT):**  $\frac{22}{4} = 5.5$  units.

Question 21:

Derive the execution order and CPU utilization using **preemptive priority scheduling** where lower numbers indicate higher priority:

Solution:

## Preemptive Priority Scheduling (Lower number = Higher Priority)

---

### Execution Logic:

- t=0:** P1 (P=3) starts. P1 rem=7.
- t=1:** P2 (P=1) arrives. **P2 preempts P1** ( $1 < 3$ ). P2 starts. P1 rem=6.
- t=3:** P3 (P=4) arrives. P3 priority (4) is lower than P2 (1). P2 continues. P2 rem=2.
- t=4:** P4 (P=2) arrives. P4 priority (2) is lower than P2 (1). P2 continues. P2 rem=1.
- t=5:** P2 runs out of time. **P2 finishes**. P2 rem=0. Ready Queue (Priorities): P4(2), P1(3), P3(4). **P4 starts** (P=2 is highest).
- t=10:** **P4 finishes**. P4 rem=0. Ready Queue: P1(3), P3(4). **P1 starts** (P=3 is highest).
- t=16:** **P1 finishes**. P1 rem=0. Ready Queue: P3(4). **P3 starts**.
- t=18:** **P3 finishes**. P3 rem=0.

### Gantt Chart (Preemptive Priority):

| P1 | P2 | P4 | P1 | P3 |
|----|----|----|----|----|
| 0  | 1  | 5  | 10 | 16 |

## Execution Order and CPU Utilization

---

**Execution Order:** P1 → P2 → P4 → P1 → P3

| Process | AT | BT | CT |
|---------|----|----|----|
| P1      | 0  | 7  | 16 |
| P2      | 1  | 4  | 5  |
| P3      | 3  | 2  | 18 |
| P4      | 4  | 5  | 10 |



### CPU Utilization:

- **Total CPU Work (Total BT):**  $7 + 4 + 2 + 5 = 18$  units.
- **Total Time Elapsed (Final CT):** 18 units.
- **Idle Time:** The Gantt chart starts at  $t = 0$  and ends at  $t = 18$  with no gaps, meaning Idle Time = 0.

$$\text{CPU Utilization} = \frac{\text{Total CPU Work}}{\text{Total Time Elapsed}} \times 100\%$$

$$\text{CPU Utilization} = \frac{18}{18} \times 100\% = 100\%$$

---

☞

---

Question 18:

Solve for the average waiting time and turnaround time using the **First Come First Serve (FCFS) algorithm** for the following processes:

Solution:

*Since Q9, Q10, Q11, and Q18 were solved using the same FCFS dataset in the first batch, I will use the dataset from Q16/Q15/Q17/Q21 to demonstrate the FCFS calculation again for this final required FCFS question.*

### FCFS Calculation (New Dataset)

---

**Dataset Used:** P1(0, 7), P2(1, 4), P3(3, 2), P4(4, 5).

**Execution Order:** P1, P2, P3, P4 (Strictly by Arrival Time)

| Process | AT | BT | CT            | TAT (CT - AT) | WT (TAT - BT) |
|---------|----|----|---------------|---------------|---------------|
| P1      | 0  | 7  | $0 + 7 = 7$   | $7 - 0 = 7$   | $7 - 7 = 0$   |
| P2      | 1  | 4  | $7 + 4 = 11$  | $11 - 1 = 10$ | $10 - 4 = 6$  |
| P3      | 3  | 2  | $11 + 2 = 13$ | $13 - 3 = 10$ | $10 - 2 = 8$  |
| P4      | 4  | 5  | $13 + 5 = 18$ | $18 - 4 = 14$ | $14 - 5 = 9$  |
| Sum     |    |    |               | 41            | 23            |

### Averages:

- **Average Turnaround Time (ATAT):**  $\frac{41}{4} = 10.25$  units.
- **Average Waiting Time (AWT):**  $\frac{23}{4} = 5.75$  units.

---

∞